

Movie Ratings Analysis

Muhammad Imran

1. Introduction

This assignment investigates movie ratings collected from five participants. Six recent popular movies were selected, and participants (5 of them) were asked to rate each movie they had seen using a 1–5 rating scale.

The achievements of this assignment are creating a database, three tables, populating the tables with constant values and then storing the data into database.

Subsequently, importing the database into R and analyse using it.

2. Questions to answer through this assignment

The assignment addresses the following questions:

1. Which movie received the highest average rating?
2. How many ratings did each movie receive?
3. How should missing ratings be handled?
4. How do individual participants differ in their average ratings?
5. How can the data be represented as a user-item matrix?

3. Database Design

The database contains three tables:

- users (user_id, name)
- movies(movie_id, title)

- ratings (user_id, movie_id, rating)

The ‘ratings’ table connects users and movies through foreign keys. The primary key of the ratings table consists of the combination of ‘user_id’ and ‘movie_id’, ensuring that a participant cannot submit multiple ratings for the same movie.

The required packages are as:

```
library(DBI)
library(RPostgres)
library(dplyr)
```

Attaching package: ‘dplyr’

The following objects are masked from ‘package:stats’:

filter, lag

The following objects are masked from ‘package:base’:

intersect, setdiff, setequal, union

```
library(tidyr)
library(ggplot2)
```

4. Connecting with the PostgreSQL

The database password is intentionally not included in the source code. However the password is “Window98”, the user has to give it on the popup.

```
library(DBI)
library(RPostgres)

con <- dbConnect(
  RPostgres::Postgres(),
  dbname = "Movie_ratings",
  host = "127.0.0.1",
  port = 5432,
  user = "postgres",
```

```
password = "Window98"  
)
```

```
dbListTables(con)
```

```
[1] "movies" "ratings" "users"
```

ensuring the connection is established and showing the tables:

```
dbListTables(con)
```

```
[1] "movies" "ratings" "users"
```

5. Retrieve Data from SQL

```
movie_ratings <- dbGetQuery(  
  con,  
  "  
  SELECT  
    u.user_id,  
    u.user_name,  
    m.movie_id,  
    m.title,  
    m.release_year,  
    r.rating  
  FROM ratings AS r  
  INNER JOIN users AS u  
    ON r.user_id = u.user_id  
  INNER JOIN movies AS m  
    ON r.movie_id = m.movie_id  
  ORDER BY u.user_id, m.movie_id  
  "  
)
```

```
#head(movie_ratings)  
knitr::kable(  
  movie_ratings,  
  caption = "Movie Ratings Data"  
)
```

Table 1: Movie Ratings Data

user_id	user_name	movie_id	title	release_year	rating
1	TOM	1	A Minecraft Movie	2025	4
1	TOM	2	F1: The Movie	2025	5
1	TOM	3	Superman	2025	4
1	TOM	4	Jurassic World Rebirth	2025	NA
1	TOM	5	How to Train Your Dragon	2025	5
1	TOM	6	The Fantastic Four: First Steps	2025	4
2	ANDER-SON	1	A Minecraft Movie	2025	5
2	ANDER-SON	2	F1: The Movie	2025	4
2	ANDER-SON	3	Superman	2025	NA
2	ANDER-SON	4	Jurassic World Rebirth	2025	4
2	ANDER-SON	5	How to Train Your Dragon	2025	5
2	ANDER-SON	6	The Fantastic Four: First Steps	2025	NA
3	JOHN	1	A Minecraft Movie	2025	3
3	JOHN	2	F1: The Movie	2025	5
3	JOHN	3	Superman	2025	5
3	JOHN	4	Jurassic World Rebirth	2025	3
3	JOHN	5	How to Train Your Dragon	2025	NA
3	JOHN	6	The Fantastic Four: First Steps	2025	4
4	AN-DREW	1	A Minecraft Movie	2025	NA
4	AN-DREW	2	F1: The Movie	2025	4
4	AN-DREW	3	Superman	2025	4
4	AN-DREW	4	Jurassic World Rebirth	2025	5
4	AN-DREW	5	How to Train Your Dragon	2025	4
4	AN-DREW	6	The Fantastic Four: First Steps	2025	5
5	PAUL	1	A Minecraft Movie	2025	4
5	PAUL	2	F1: The Movie	2025	NA
5	PAUL	3	Superman	2025	3

user_id	user_name	movie_id	title	release_year	rating
5	PAUL	4	Jurassic World Rebirth	2025	4
5	PAUL	5	How to Train Your Dragon	2025	5
5	PAUL	6	The Fantastic Four: First Steps	2025	4

6. Statistical Analysis of the Data

```
summary(movie_ratings)
```

```

      user_id      user_name      movie_id      title      release_year
Min.   :1  Length   :30  Min.   :1.0  Length   :30  Min.   :2025
1st Qu.:2  N.unique  : 5  1st Qu.:2.0  N.unique  : 6  1st Qu.:2025
Median :3  N.blank   : 0  Median :3.5  N.blank   : 0  Median :2025
Mean    :3  Min.nchar: 3  Mean    :3.5  Min.nchar: 8  Mean    :2025
3rd Qu.:4  Max.nchar: 8  3rd Qu.:5.0  Max.nchar:31  3rd Qu.:2025
Max.    :5                                     Max.    :6.0                                     Max.    :2025

      rating
Min.   :3.00
1st Qu.:4.00
Median :4.00
Mean    :4.25
3rd Qu.:5.00
Max.    :5.00
NAs     :6

```

7. Missing Data

Not every participant had seen every movie. Therefore, missing ratings

are represented by NULL in PostgreSQL and become NA in R.

These values are not treated as zero because zero is not part of the 1–5 rating scale.

First, the number of missing ratings is calculated:

```
sum(is.na(movie_ratings$rating))
```

```
[1] 6
```

This is telling that there are 6 missing/NULL values in the table.

```
mean(is.na(movie_ratings$rating)) * 100
```

```
[1] 20
```

This is showing that 20% values are missing.

```
library(dplyr)
```

```
movie_rating_counts <- movie_ratings %>%  
  group_by(title) %>%  
  summarise(  
    ratings_received = sum(!is.na(rating)),  
    missing_ratings = sum(is.na(rating))  
  )
```

```
movie_rating_counts
```

```
# A tibble: 6 x 3  
  title                ratings_received missing_ratings  
  <chr>                  <int>          <int>  
1 A Minecraft Movie           4              1  
2 F1: The Movie              4              1  
3 How to Train Your Dragon    4              1  
4 Jurassic World Rebirth      4              1  
5 Superman                   4              1  
6 The Fantastic Four: First Steps 4              1
```

8. Ratings by Movie

```

movie_summary <- movie_ratings %>%
  group_by(title) %>%
  summarise(
    number_of_ratings = sum(!is.na(rating)),
    average_rating = mean(rating, na.rm = TRUE),
    minimum_rating = min(rating, na.rm = TRUE),
    maximum_rating = max(rating, na.rm = TRUE)
  ) %>%
  arrange(desc(average_rating))
movie_summary

```

```

# A tibble: 6 x 5
  title                number_of_ratings average_rating minimum_rating maximum_rating
  <chr>                  <int>          <dbl>          <int>          <int>
1 How to Train Y~         4            4.75            4            5
2 F1: The Movie           4            4.5             4            5
3 The Fantastic ~        4            4.25            4            5
4 A Minecraft Mo~        4            4              3            5
5 Jurassic World~        4            4              3            5
6 Superman               4            4              3            5

```

9. Highest-Rated Movie

```

movie_summary %>%
  slice_max(
    order_by = average_rating,
    n = 1
  )

```

```

# A tibble: 1 x 5
  title                number_of_ratings average_rating minimum_rating maximum_rating
  <chr>                  <int>          <dbl>          <int>          <int>
1 How to Train Y~         4            4.75            4            5

```

10. Calculate ratings (Average) by participant

```
user_summary <- movie_ratings %>%
  group_by(user_name) %>%
  summarise(
    number_of_ratings = sum(!is.na(rating)),
    average_rating = mean(rating, na.rm = TRUE)
  )
user_summary
```

```
# A tibble: 5 x 3
  user_name number_of_ratings average_rating
  <chr>          <int>          <dbl>
1 ANDERSON             4             4.5
2 ANDREW               5             4.4
3 JOHN                 5             4
4 PAUL                 5             4
5 TOM                  5             4.4
```

11. Generating a user/movie matrix

```
library(dplyr)
library(tidyr)

# 1. Reshape and ensure all columns render cleanly
rating_matrix <- movie_ratings %>%
  select(user_id, user_name, title, rating) %>%
  pivot_wider(
    id_cols = c(user_id, user_name),
    names_from = title,
    values_from = rating,
    values_fill = NA # Replaces unrated cells explicitly with NA
  )

# 2. Print all columns in console without truncation
as.data.frame(rating_matrix)
```

```
user_id user_name A Minecraft Movie F1: The Movie Superman
```

1	1	TOM	4	5	4
2	2	ANDERSON	5	4	NA
3	3	JOHN	3	5	5
4	4	ANDREW	NA	4	4
5	5	PAUL	4	NA	3

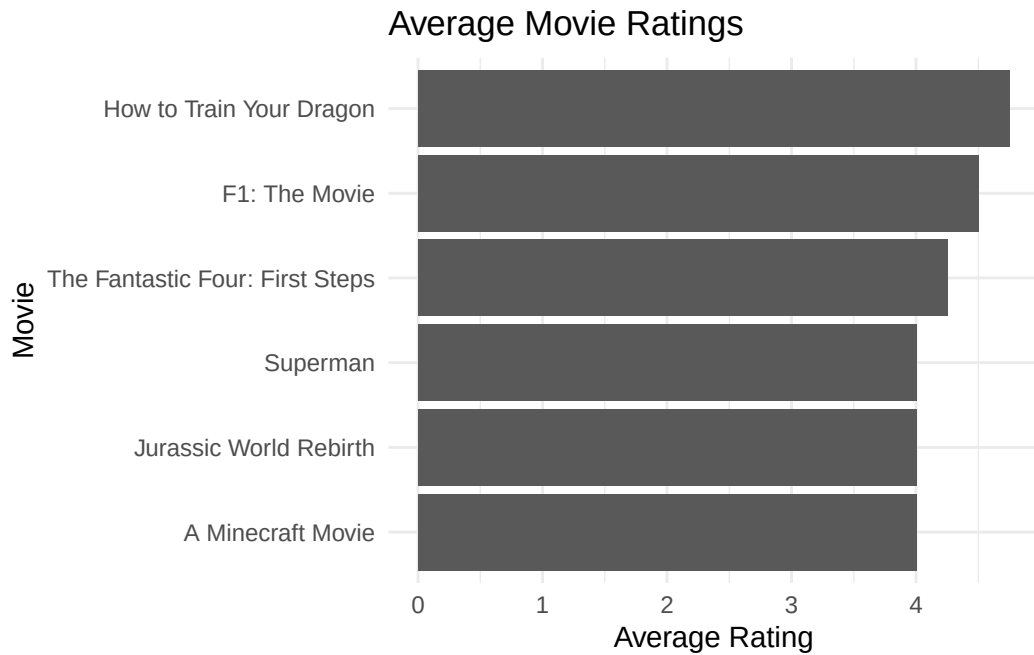
	Jurassic World Rebirth	How to Train Your Dragon
1	NA	5
2	4	5
3	3	NA
4	5	4
5	4	5

	The Fantastic Four: First Steps
1	4
2	NA
3	4
4	5
5	4

12. Creating visualizations

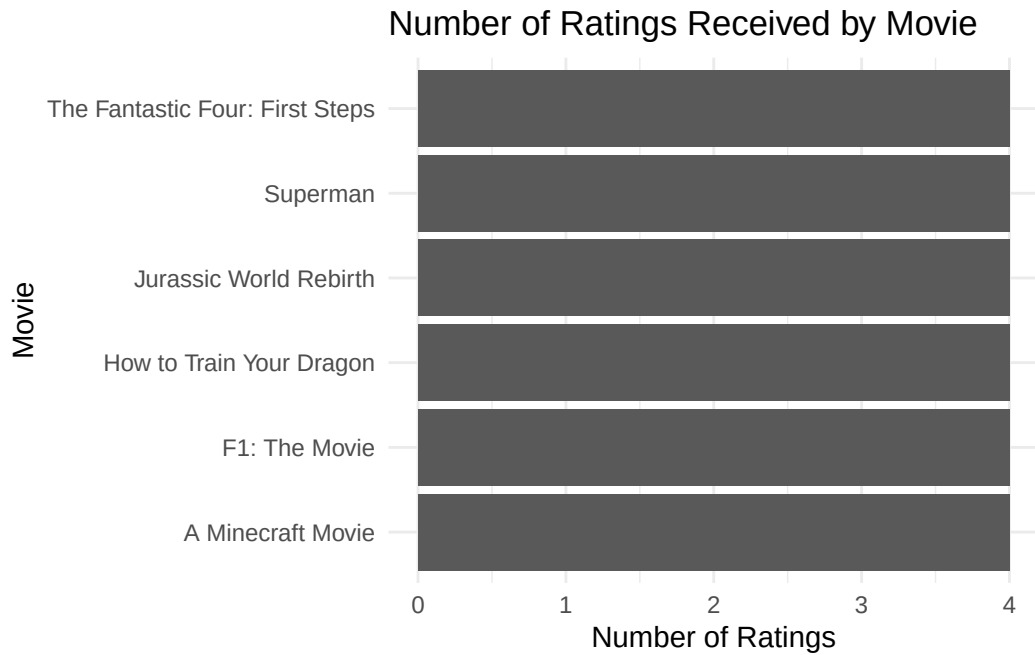
```
library(ggplot2)

ggplot(
  movie_summary,
  aes(
    x = reorder(title, average_rating),
    y = average_rating
  )
) +
  geom_col() +
  coord_flip() +
  labs(
    title = "Average Movie Ratings",
    x = "Movie",
    y = "Average Rating"
  ) +
  theme_minimal()
```



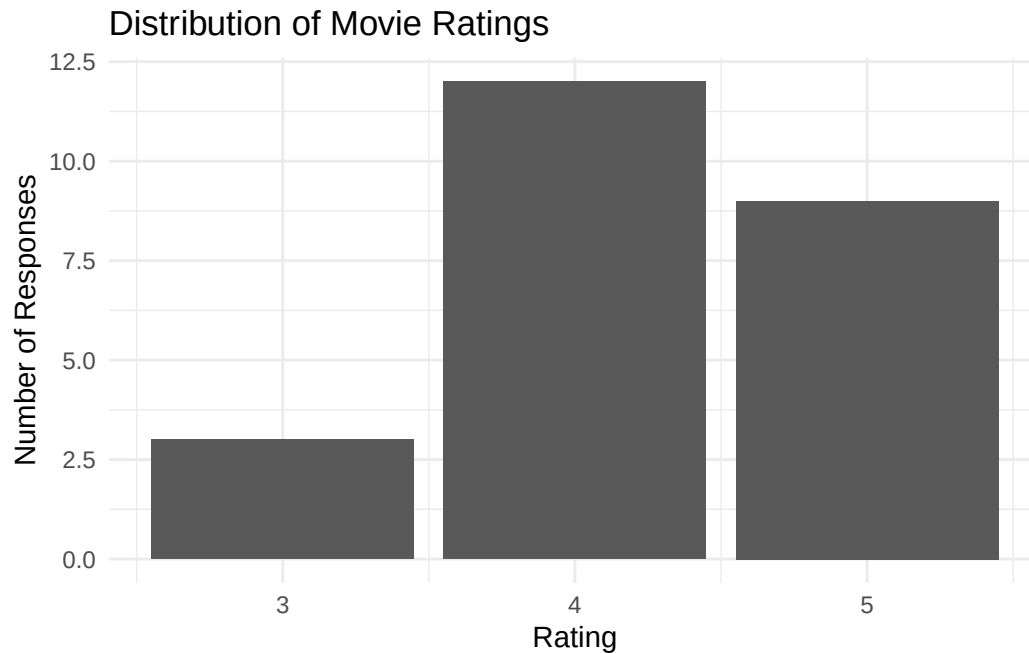
Visualize number of ratings

```
ggplot(  
  movie_summary,  
  aes(  
    x = reorder(title, number_of_ratings),  
    y = number_of_ratings  
  )  
) +  
  geom_col() +  
  coord_flip() +  
  labs(  
    title = "Number of Ratings Received by Movie",  
    x = "Movie",  
    y = "Number of Ratings"  
  ) +  
  theme_minimal()
```



Optional: Distribution of ratings

```
ggplot(  
  movie_ratings %>% filter(!is.na(rating)),  
  aes(x = rating)  
) +  
  geom_bar() +  
  scale_x_continuous(breaks = 1:5) +  
  labs(  
    title = "Distribution of Movie Ratings",  
    x = "Rating",  
    y = "Number of Responses"  
  ) +  
  theme_minimal()
```



13. Interpretation

The average-rating analysis identifies which movies were most highly rated by the participants. The number of ratings is also important because not every participant had seen every movie. Therefore, averages should be interpreted together with the number of observations contributing to each average. Missing ratings were not converted to zero. Instead, they were treated as missing observations because a participant who had not seen a movie could not reasonably provide a rating.

14. Limitations

This dataset has several limitations.

First, the sample size is small because the assignment requires only five participants. Therefore, the results cannot be generalized to the broader population.

Second, participants may have different movie preferences and levels of familiarity with the selected movies.

Third, missing ratings may affect comparisons because some movies may have been seen by more participants than others.

Finally, the dataset is intended for demonstrating SQL and R concepts rather than producing statistically representative estimates of movie quality.

15. Conclusion

This Assignment demonstrates a complete workflow from data collection to database storage and statistical analysis. The movie ratings were stored using a normalized relational database containing users, movies, and ratings tables. SQL joins were then used to retrieve an analysis-ready dataset, which was imported into R for data cleaning, missing-data handling, summary statistics, and visualization. The resulting user-item matrix also demonstrates how a simple movie rating dataset can serve as the foundation for recommender-system methods such as collaborative filtering.